

Malhotra Automobiles

Product Requirements Document (PRD)

Professional website + product request system + service booking + realtime support + admin platform

Malhotra Automobiles — Product Requirements Document (PRD)

1. Product Overview

Malhotra Automobiles will be a professional, mobile-first automobile business platform combining a public business website, product catalogue/cart, service booking, customer requests, real-time customer support, and an owner/admin dashboard.

The initial business model is request-based rather than online payment: customers select products and/or services, submit a request, receive confirmation/status updates, and physically visit the business for fulfilment and payment.

2. Goals

- Establish a trustworthy professional online presence.
- Let customers browse products and add them to a cart.
- Let customers request products without requiring online payment.
- Let customers browse services and book available time slots.
- Allow product requests and service bookings to be combined into one visit request.
- Notify the business owner and customer by email.
- Provide real-time customer-to-business chat.
- Provide an AI assistant using Gemini for common questions and guided assistance.
- Give the owner one secure dashboard to manage products, services, slots, requests, customers and conversations.
- Build a foundation that can later support payments, inventory, CRM, WhatsApp and advanced analytics.

3. Non-Goals for MVP

- Online payment/checkout.
- Delivery/shipping.
- Complex marketplace/multi-vendor functionality.
- Public product reviews requiring moderation workflows.
- Full accounting/invoicing.
- Native mobile apps.

4. Target Users

Customer

A local vehicle owner looking for products, accessories or automobile services.

Business Owner/Admin

Malhotra Automobiles staff responsible for catalogue, availability, requests and customer communication.

Future Staff Role

A limited staff role may later be introduced with restricted permissions.

5. Customer Experience

Landing Page

Sections:

- Header/navigation
- Hero with primary CTAs: Browse Products and Book a Service
- Featured products
- Services
- Why choose Malhotra Automobiles
- Offers/promotions
- Testimonials
- About/business information
- Location, opening hours and contact
- Footer
- Persistent chat CTA

Products

Customers can:

- Browse categories.
- Search products.
- Filter and sort.
- View product details.
- Select quantity.
- Add products to cart.

Product fields:

- Name
- Brand
- Category
- SKU/part number
- Price
- Optional discount
- Description
- Specifications
- Vehicle compatibility
- Images
- Stock/availability status
- Published/hidden state
- Featured flag

Cart

The cart should show:

- Product image/name
- Quantity controls

- Unit price
- Estimated subtotal/total
- Remove item
- Continue shopping
- Confirm request

No payment is collected in MVP.

Request Submission

Customer provides:

- Name
- Mobile number
- Email
- Vehicle make/model
- Vehicle registration number (optional/configurable)
- Notes

A request may contain:

- Products only
- Service only
- Both products and service

On submission, generate a unique request ID.

Service Booking

Customers can:

- Browse services.
- Open service details.
- Select date.
- See only available slots.
- Select a slot.
- Add optional products.
- Submit request.

Service fields:

- Name
- Description
- Estimated duration
- Starting price/price display
- Image
- Active/inactive

Availability

Owner controls available slots. Customers must never be able to book a slot that is already unavailable.

Slot states:

- Available
- Held/processing

- Booked
- Blocked/closed
- Cancelled

The backend must perform an authoritative availability check at booking time to prevent double booking.

Confirmation / Status

After submission:

- Show request ID.
- Show selected products/services.
- Show selected date/time if applicable.
- Show current status.

Suggested statuses:

1. Submitted
2. Under Review
3. Accepted
4. Rejected
5. Ready for Visit
6. Completed
7. Cancelled

Customer-facing wording can be simplified while internal statuses remain precise.

6. Customer Authentication

MVP may allow browsing and request submission without an account, but Supabase Auth should be implemented so accounts can be enabled cleanly.

Authenticated customers can eventually see:

- Profile
- Vehicles
- Previous requests
- Bookings
- Conversations
- Status history

Admin authentication is mandatory.

7. Real-Time Chat

Human Chat

Customer can start a conversation with Malhotra Automobiles. Admin sees incoming conversations in the dashboard and can reply in real time.

Requirements:

- Conversation creation
- Message sending/receiving
- Read/unread state
- Conversation status
- Admin assignment (future)

- Timestamps
- Basic abuse/rate controls

AI Assistant

Gemini provides:

- Business FAQ answers
- Website navigation help
- Service explanations
- Product discovery assistance
- Booking guidance

Gemini must not invent product availability, pricing or booking availability.

For factual business data, the backend should retrieve relevant data from Supabase and provide it as controlled context/tools to the model.

Suggested escalation:

AI unable to answer -> offer human support -> transfer/continue in human chat.

The Gemini API key must remain server-side.

8. Email Notifications

Use Resend or equivalent transactional email provider.

Owner notifications:

- New request
- New booking/request requiring review
- New customer chat (optional configurable)
- Important cancellations

Customer notifications:

- Request received
- Request accepted
- Request rejected
- Booking confirmed
- Booking changed/cancelled
- Optional reminder

Email templates should include request ID, customer information, service details, products, date/time and clear next steps.

9. Admin Dashboard

Dashboard Overview

Display:

- Pending requests
- Today's bookings
- Upcoming bookings
- Product count
- Unread conversations
- Recent activity

Product Management

Owner can:

- Create/edit/delete products
- Upload images
- Manage categories
- Publish/unpublish
- Mark featured
- Change price
- Change availability
- Manage compatibility/specifications

Service Management

Owner can:

- Create/edit/delete services
- Set duration
- Set pricing display
- Upload images
- Activate/deactivate services

Slot Management

Owner can:

- Create slots
- Create recurring availability patterns
- Block dates/times
- Mark slots unavailable
- View bookings
- Modify availability

Request Management

Owner can:

- View requests
- Filter by status/date
- Open complete request details
- Accept/reject
- Change status
- Cancel
- Mark completed
- View customer and vehicle information
- View requested products/services

Customer Management

Owner can:

- Search customers
- View customer profile

- View request history
- View booking history
- View vehicles
- Open conversations

Chat Management

Owner can:

- View conversations
- Reply in real time
- Mark conversations read
- Close/archive conversations
- Take over from AI

Settings

- Business profile
- Address/contact
- Opening hours
- Email settings
- Booking rules
- Slot duration/defaults
- Notification preferences
- Admin account settings

10. Roles & Permissions

Admin

Full access.

Customer

Access only to their own account/request/conversation data.

All sensitive operations must be protected by backend authorization and Supabase RLS. Never trust client-side role checks alone.

11. Recommended Technology Stack

Frontend

- React
- TypeScript
- Vite
- Tailwind CSS
- shadcn/ui
- React Router
- TanStack Query
- Zustand

Backend

- Node.js

- Express
- TypeScript
- Zod
- Helmet
- CORS
- Rate limiting

Supabase

- PostgreSQL
- Supabase Auth
- Supabase Storage
- Supabase Realtime
- Row Level Security

AI

- Google Gemini API

Email

- Resend

Testing

- Vitest
- React Testing Library
- Playwright

Tooling

- Git
- GitHub
- ESLint
- Prettier
- Environment-based configuration

12. High-Level Architecture

Customer/Admin React app

-> Node.js/Express API

-> Supabase PostgreSQL/Auth/Storage/Realtime

Node.js API also integrates with:

- Gemini API
- Resend

Architecture principles:

- Supabase database is the source of truth.
- Backend owns sensitive business logic.
- Gemini is an intelligence layer, not a database.
- Frontend never contains secrets.
- RLS protects database records.

- Server validates all important actions.

13. Core Data Model

Recommended entities:

users/profiles

- id
- auth_user_id
- role
- name
- phone
- email
- created_at
- updated_at

vehicles

- id
- customer_id
- make
- model
- year
- registration_number
- notes

categories

- id
- name
- slug
- description
- image
- active

products

- id
- category_id
- name
- slug
- brand
- sku
- price
- discount_price
- description
- specifications
- compatibility

- availability_status
- featured
- published
- created_at
- updated_at

product_images

- id
- product_id
- storage_path
- sort_order

services

- id
- name
- slug
- description
- duration_minutes
- price
- image
- active

availability_slots

- id
- date
- start_time
- end_time
- status
- capacity
- notes

requests

- id
- request_number
- customer_id
- vehicle_id
- status
- notes
- estimated_total
- created_at
- updated_at

request_items

- id
- request_id

- product_id
- quantity
- unit_price_snapshot

request_services

- id
- request_id
- service_id
- slot_id
- price_snapshot
- duration_snapshot

conversations

- id
- customer_id
- status
- assigned_admin_id
- ai_enabled
- created_at
- updated_at

messages

- id
- conversation_id
- sender_type
- sender_id
- content
- created_at
- read_at

notifications

- id
- recipient_id
- type
- title
- message
- read_at
- created_at

audit_logs

- id
- admin_id
- action
- entity_type
- entity_id

- metadata
- created_at

Use immutable price snapshots in request items/services so historical requests do not change when catalogue prices change.

14. API Areas

Backend should expose versioned endpoints, e.g. `/api/v1``.

Areas:

- auth/profile
- products
- categories
- services
- availability
- requests
- bookings
- customers
- chat
- notifications
- admin
- AI assistant

Exact endpoint naming should be finalized during implementation.

15. Business Rules

1. Only published/active products appear publicly.
2. Only active services appear publicly.
3. Only available slots can be selected.
4. Slot availability must be checked transactionally/server-side.
5. A request does not equal a completed sale.
6. Product prices are snapshotted when a request is submitted.
7. Owner acceptance is required before a request is considered confirmed.
8. Rejected/cancelled requests cannot remain as active bookings.
9. Admin-only operations require authenticated admin authorization.
10. Customers cannot access other customers' requests or conversations.
11. AI cannot claim inventory, price or availability without current backend data.
12. Product deletion should preferably be soft deletion/unpublish when referenced historically.
13. All timestamps should be stored consistently (UTC) and displayed in the business/customer timezone.

16. UX Requirements

- Mobile-first responsive design.
- Fast initial page load.
- Clear CTAs.
- Strong automobile visual identity.
- Professional rather than overly flashy.

- Accessible contrast and keyboard navigation.
- Loading, empty and error states for all async operations.
- Toast/inline feedback for important actions.
- Confirmation dialogs for destructive admin actions.
- Skeleton loaders for product/admin data.
- Persistent cart indicator.
- Persistent chat button.

17. Suggested Visual Direction

Brand personality:

- Professional
- Reliable
- Automotive
- Local/trustworthy
- Modern

Visual language:

- Strong typography
- Clean cards
- Automotive imagery
- Structured grids
- Subtle motion
- High-quality product photography
- Clear status indicators
- Dashboard with dense but readable data

The final color palette should be agreed with the client and can be derived from the Malhotra Automobiles logo/signage if available.

18. SEO

Public pages should support:

- Page titles
- Meta descriptions
- Open Graph metadata
- Semantic HTML
- Clean URLs/slugs
- Product/service structured data where appropriate
- Local business structured data
- Sitemap
- robots.txt

19. Security

- HTTPS in production.
- Secrets only in environment variables.

- Supabase service-role key never exposed to browser.
- Backend input validation with Zod.
- Helmet security headers.
- Strict CORS configuration.
- Rate limiting.
- Authentication and authorization on every protected operation.
- Supabase RLS.
- Secure upload rules.
- Audit logging for important admin actions.
- Sanitized/validated chat input.
- Abuse controls for public endpoints.

20. Testing Strategy

Unit

Business logic, validators, utilities.

Integration

API/database operations, booking conflict logic, authorization.

E2E

Critical flows:

1. Browse product -> cart -> submit request.
2. Browse service -> choose slot -> submit booking.
3. Service + products -> combined request.
4. Admin accepts/rejects request.
5. Customer sees updated status.
6. Customer sends chat -> admin replies.
7. AI answers using current business data.

21. Deployment

Suggested:

- React frontend: Vercel or equivalent.
- Node.js API: Render/Railway/Fly.io or equivalent.
- Database/Auth/Storage/Realtime: Supabase.
- Email: Resend.
- AI: Gemini API.

Use separate development and production environments.

22. Observability

At minimum:

- Server error logging
- API request/error logs
- Admin audit logs

- Email delivery/error tracking
- AI request/error monitoring
- Basic uptime monitoring

23. Future Roadmap

Phase 2:

- Customer accounts/history
- Vehicle profiles
- AI-human handoff improvements
- Reviews
- Service reminders
- Better analytics

Phase 3:

- Online payment
- Inventory quantities
- Coupons
- Invoices
- WhatsApp integration
- Loyalty system
- CRM
- Automated service reminders

24. MVP Acceptance Criteria

The MVP is complete when:

- Owner can log into admin dashboard.
- Owner can add/edit/publish products.
- Customer can browse and search products.
- Customer can add/remove/update cart items.
- Customer can submit a product request.
- Owner receives notification.
- Customer receives request acknowledgement.
- Owner can create/manage service slots.
- Customer can see and select only available slots.
- Customer can submit a service booking.
- Customer can combine products and service in one request.
- Owner can accept/reject/update requests.
- Customer can see request status.
- Real-time human chat works.
- Gemini assistant can answer controlled business questions.
- Customer data is protected.
- Admin routes are protected.
- Mobile and desktop layouts are responsive.

- Core customer/admin flows have automated tests.

25. Implementation Principles for Claude Code

- Build incrementally by feature, not as one giant generation.
- Keep frontend and backend responsibilities clearly separated.
- Create database migrations before dependent UI.
- Implement RLS and authorization early.
- Use typed API contracts.
- Never hard-code business data that belongs in Supabase.
- Never expose API secrets.
- Keep components reusable.
- Avoid premature abstraction.
- Write tests for booking conflicts and authorization.
- Maintain a clear README and environment-variable documentation.
- Do not add online payment until explicitly requested.